

MACRO PROPERTY SERVICES

Architecture Document

Employee App & Admin Dashboard — System Design

Next.js 16 · npm workspaces

Supabase (Postgres + Auth + RLS)

Firebase Firestore (chat)

ImageKit

01 System Overview

Two independently deployable Next.js applications share one backend and one code package — a monorepo, not a single combined app, so each side ships and scales on its own schedule while security-sensitive logic (permission checks, geofencing, the Supabase/Firebase clients) isn't duplicated and can't drift between them.

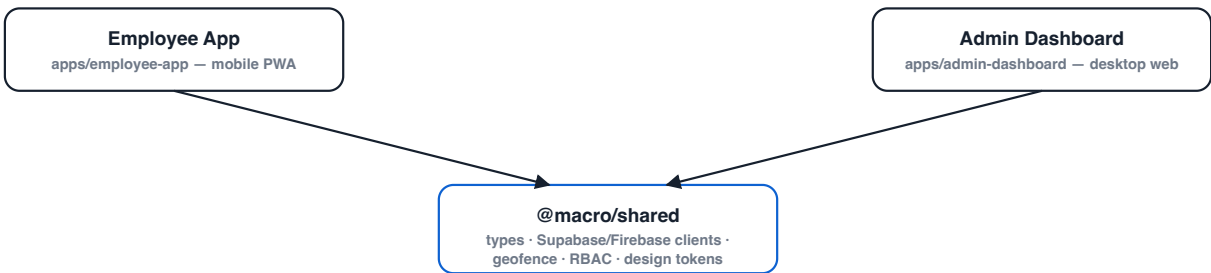


Fig. 1 — Both apps import the same `packages/shared` code — types, the Supabase/Firebase client factories, the geofence formula, and the RBAC helpers are defined once. How each app then talks to Supabase and Firebase is covered in Fig. 2.

Employee App

Mobile-first PWA for field employees: attendance, checklists, audits, and company communication. Installable to a home screen; a minimal service worker keeps the login shell available offline.

Admin Dashboard

Desktop web console for managing companies, sites, employees, roles/permissions, and reviewing attendance/audits/checklists/communication across the business.

02 Tech Stack

Chosen for one reason each — not a maximal stack, and nothing here is used for more than one job.

LAYER	CHOICE	WHY
Framework	Next.js 16, App Router	Server Components/Actions remove the need for a hand-rolled REST/GraphQL API layer between the apps and Supabase.
Monorepo	npm workspaces	Already-installed tooling; <code>apps/*</code> + <code>packages/shared</code> , no extra global CLI to adopt.
Database	Supabase Postgres	Row-Level Security enforces access control at the data layer itself, not just in application code.
Auth	Supabase Auth	Cookie-based sessions via <code>@supabase/ssr</code> , shared by both apps' middleware.
Scheduling	pg_cron	Runs <code>generate_due_checklists()</code> every 5 minutes — no separate worker process to deploy/monitor.
Live chat	Firebase Firestore	Real-time listeners out of the box; Postgres is not built for that access pattern. Used for chat only — nothing else lives here.
Push notifications	Firebase Cloud Messaging	Web Push for new-message alerts, sent recipient-by-recipient (never broadcast) via a Supabase Edge Function.
Image hosting	ImageKit	Handles all audit/checklist/communication photo uploads and CDN delivery. (Supabase Storage buckets exist from an earlier iteration but are currently unused.)
Styling	Tailwind CSS + shared design tokens	One token set (<code>packages/shared/src/tokens</code>) so both apps render the same palette/type scale.

03 Backend Architecture

Almost everything is Postgres. Firebase is deliberately scoped to the one thing Postgres isn't built for — live message delivery — and every other read/write goes straight through Supabase from each app's own Server Components and Server Actions.

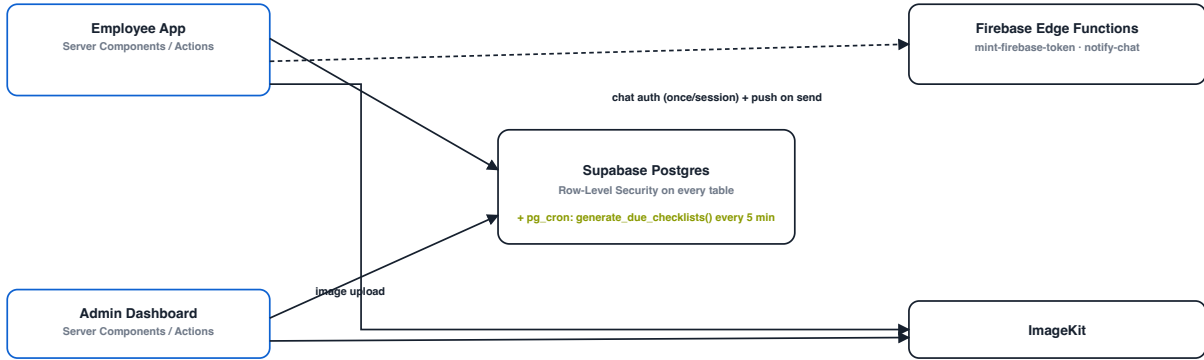


Fig. 2 — Postgres is the single source of truth for everything except live chat messages; `pg_cron` runs inside Postgres itself, not as an external caller. Firestore access is gated by a Firebase custom token minted from the caller's real Supabase session, not a separate login.

04 Data Model

Core Postgres tables and how they relate. All fourteen tables have Row-Level Security enabled.

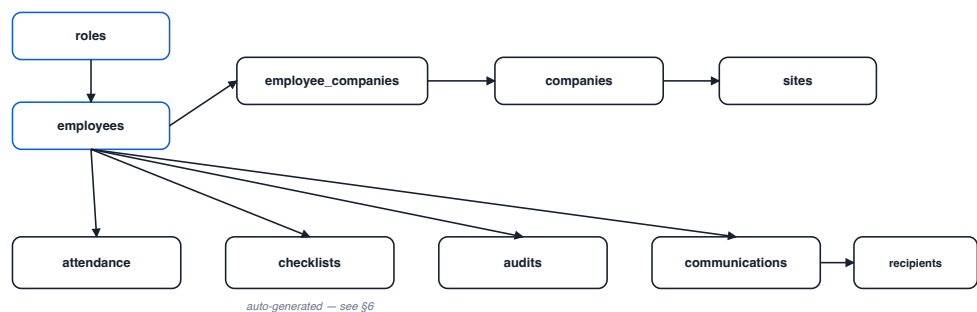


Fig. 3 — Every activity table carries its own `employee_id` / `company_id` / `site_id` for direct scoping (not drawn individually here for clarity). `checklists` rows are generated automatically from a standing `checklist_assignments` + `checklist_templates` pairing — see §6. `communications` is metadata only; message content lives in Firestore.

Notable tables

TABLE	PURPOSE
<code>roles</code>	Configurable Access Roles. Permissions stored as one <code>jsonb</code> column (dashboard areas × app areas × function flags) rather than fully normalized, for fidelity with the Roles & Access UI and fast reads.
<code>employee_companies</code>	Join table — an employee can belong to more than one company/site at once; attendance is tracked per site independently.
<code>checklist_assignments</code> → <code>checklists</code>	A standing assignment (no date) is expanded into a dated <code>checklists</code> row automatically by the scheduler, deep-copying the template's areas at that moment (not a live reference).
<code>communications</code> / <code>communication_recipients</code>	Postgres holds the thread's metadata and its fixed recipient list; actual messages live in Firestore, keyed by the same id.
<code>device_tokens</code>	FCM web-push tokens per employee, used by the <code>notify-chat</code> function to push to exactly a thread's other recipients.
<code>notification_dismissals</code>	Per-employee "seen/dismissed" markers for the Home screen notification bell.

05 Security Model

Access control is enforced at three independent layers — losing any one of them doesn't expose data, because the others still hold.

1 · Row-Level Security (Postgres)

Every table has RLS enabled. Policies call small

`SECURITY DEFINER` helper functions —
`is_admin()`, `user_company_ids()`,
`is_communication_recipient()`,
`has_checklist_review_access()`,
`has_checklist_images_only_access()` —

instead of inlining subqueries per policy, which avoids RLS self-reference recursion and keeps the access rule defined once.

2 · Firestore Security Rules

Independent of Postgres RLS — chat lives in a different database entirely. Each `conversations/{id}` doc carries an `employeeIds` array; both the doc and its `messages` subcollection require the caller's uid to be in that array (or an admin claim), checked on every read/write, not just at doc creation.

3 · RBAC permission schema

A fixed `RolePermissions` shape (dashboard areas × app areas, each with specific function flags) drives both which UI a role even renders and, server-side, which routes middleware allows — the UI hiding a button is a convenience, not the actual gate.

4 · Geofencing

Clock in/out location is re-validated server-side (Haversine distance, default 20m radius) against the site's coordinates regardless of what the client reports — a client-side bypass can't fake a valid geofence result.

Why Admin doesn't automatically see everything

Communications were deliberately hardened to be participant-only, including for Admin — an early design had a blanket `is_admin()` policy granting full read access to every thread for oversight, which turned out to let Admin-flagged accounts see private 1:1 conversations they had no part in. That blanket policy was removed; Admin can still start a thread with anyone (and is auto-added as a participant of threads they create), and can still close/reopen or delete any thread for moderation — but can no longer silently read one they weren't added to.

Known caveat — client-side navigation cache. Next.js keeps a client-side Router Cache of previously-fetched pages, independent of server-side `force-dynamic`. Both apps set `staleTimes.dynamic: 0` so every in-app navigation always re-fetches live, correctly-scoped data rather than reusing a page rendered under a since-changed permission set.

06 Key Server-Side Workflows

Firestore auth bridge (chat)

Chat needs Firestore, but there's only one login (Supabase). The `mint-firebase-token` Edge Function verifies the caller's real Supabase session, looks up their role's `is_admin` flag, and hand-signs (RS256, via Web Crypto — no firebase-admin SDK needed in Deno) a Firebase custom token with `uid = the Supabase user's id` and claims `{ employeeId, admin }`. The client signs into Firebase with that token once per session — no second password, and the Firestore rules above can trust `request.auth.uid` as the real employee id.

Scheduled checklist generation

`generate_due_checklists()` runs every 5 minutes via `pg_cron`. For each standing `checklist_assignments` row, it checks the company's visit frequency (weekly/fortnightly/custom) and days against "today" computed in the `Australia/Sydney` timezone, and inserts today's `checklists` row if one doesn't already exist for that employee/template/date. It fires as soon as possible after midnight rather than at an admin-configured time — simpler to reason about, and nothing silently fails to generate because a time field was left blank.

Recipient-scoped push notifications

Sending a chat message calls `notify-chat`, which re-verifies the caller is actually a participant of that conversation, reads only the *other* recipients' `device_tokens`, and pushes to exactly those via FCM's HTTP v1 API — never a broadcast. Tokens that FCM reports as unregistered/invalid are pruned from the table automatically.

Image uploads

Audit, checklist, and communication photos upload through a Server Action that calls `uploadImageToImageKit()` — the ImageKit private key stays server-only, and the client only ever receives back a public CDN URL to store/display.

07 Deployment & Environments

Each app deploys independently and reads its own environment file — there is no shared runtime between them beyond the backend they both call.

VARIABLE	WHERE	NOTES
NEXT_PUBLIC_SUPABASE_URL / _ANON_KEY	Both apps	Public — safe in the browser bundle; RLS is the real gate.
SUPABASE_SERVICE_ROLE_KEY	Admin Dashboard only	Server-only, bypasses RLS — used solely for provisioning new employee Auth users.
NEXT_PUBLIC_FIREBASE_*	Both apps	Firebase project config; chat degrades to a "not configured" state if blank rather than crashing.
NEXT_PUBLIC_FIREBASE_VAPID_KEY	Employee App	Enables Web Push registration; push notifications simply stay off without it.
IMAGEKIT_PRIVATE_KEY	Both apps	Server-only.
FIREBASE_CLIENT_EMAIL / _PRIVATE_KEY	Supabase Edge Function secrets	Service-account credentials for mint-firebase-token and notify-chat — never in either app's own env.

Independent deploys. apps/employee-app and apps/admin-dashboard are meant to be deployed as two separate projects (e.g. two Vercel projects pointed at the same repo's two subfolders) — a change to one doesn't require redeploying the other.